NIDS Attack Lab

Generating every attack the Counselor NIDS flags — to test detection and push Snort rule-trust to 0.91–0.97

Authorised use only. Run these against a host **you own**, on your **own** network, to test your own NIDS. Attacking systems you do not own is illegal.

1. Setup — where to attack from

The NIDS listens on one interface (**wlan0**). It can only judge traffic that **crosses that interface**. Two consequences decide your test setup:

• **You cannot attack this laptop from itself.** Traffic to your own IP (172.20.10.3) is routed over loopback (`lo`), never across wlan0, so the NIDS never sees it.

• **Target a different device you own** on the same hotspot — a second laptop, a Raspberry Pi, or a VM with a bridged adapter — reachable at a `172.20.10.x` address. Attack it from this laptop, or attack this laptop from it. Either way the traffic crosses wlan0 and is judged.

• **Do not target the gateway** (172.20.10.1). That is your phone / ISP equipment, not yours to attack, and flooding it drops your hotspot.

What the NIDS reads is the **behaviour** of each connection (who, ports, packets, bytes, duration, speed, TCP flags) — never the contents. So an attack is detected by its shape, not by reading its payload.

2. Every attack type and what flags it

Each command maps to the specialist detector it should trip and the Snort rule category that should fire alongside it. When both agree, that is the strongest alert — and it is what raises rule-trust.

Command	What it does	AI detector	Snort rule
scan	TCP port scan (nmap -sS -p-)	live_access	port_scan (122:x)
pingsweep	ICMP host sweep of the subnet	— (recon)	ICMP sweep
synflood	TCP SYN flood (hping3, rate-limited)	live_dos	SYN-flood rule
udpflood	UDP flood (hping3)	live_dos	—
icmpflood	ICMP (ping) flood (hping3)	live_dos	—
slowloris	Slow-HTTP DoS (slowhttptest)	live_dos	request-flood rule
httpflood	HTTP GET flood (curl loop)	live_dos	—
sshbrute	SSH password guessing (nmap NSE)	live_access	brute-force rule
ftpbrute	FTP password guessing (nmap NSE)	live_access	brute-force rule
web	Full web-attack scan (nikto)	live_access	web rules
sqli	SQL-injection requests (curl)	—	SQL-injection rule
xss	Cross-site-scripting requests (curl)	—	XSS rule
traversal	Path-traversal requests (curl)	—	path-traversal rule
botnet	C2 beaconing simulation (steady check-ins)	live_bot	—

Notes. **sqli / xss / traversal / web / slowloris** need a web server running on the target. **sshbrute / ftpbrute** need those services open on the target. **botnet** is recognised by behaviour, not a signature, so point it at a host you own that plays the pretend control server; it may not always flag. The port scan often makes the AI *disagree* (it was trained on few scan examples) — useful to see, but it lowers that rule's trust rather than raising it.

3. Pushing rule-trust to 0.91–0.97

Trust is **earned, not set**. Each Snort rule's trust is a Beta posterior that starts at the 0.90 prior (weight 20) and moves only when the rule fires and the AI's confident verdict is compared with it:

```
trust = (0.9 x 20 + agreements) / (20 + agreements + disagreements)
      = (18 + agreements)   / (20 + agreements + disagreements)
```

It **rises only on agreements** (Snort and the AI both say attack). Disagreements pull it down. From a clean 0.90, the agreements you need:

Target trust	0.91	0.92	0.93	0.95	0.96	0.97
Agreements on that rule	~3	~5	~9	~20	~30	~47

Reliable path: use **floods** (synflood / slowloris). live_dos is the strongest detector, so it agrees with Snort's flood rule and trust climbs steadily — run the flood a few times to accumulate agreements. Avoid relying on scans, which tend to produce disagreements.

4. Running it

The toolkit lives at `~/nids-attack-lab/nids-attack-lab.sh`. Start the NIDS first (`./start.sh live wlan0`), then:

```
cd ~/nids-attack-lab
./nids-attack-lab.sh watch                 # baseline: alerts + rule trust
./nids-attack-lab.sh synflood 172.20.10.7  # TARGET = a device you own
./nids-attack-lab.sh synflood 172.20.10.7  # repeat to build agreements
./nids-attack-lab.sh watch                 # see detection + trust climbing

./nids-attack-lab.sh all 172.20.10.7       # scan -> flood -> web -> sshbrute,
                                           # with before/after trust
```

Reading the results on the dashboard: the **Alerts** page lists each flagged connection with what the AI and the rule checker said (a flood shows as “Flooding attack” caught by both = highest confidence). The **AI detectors** page shows the per-rule trust table moving. A normal-but-unusual connection now also appears as “Possible new or unknown attack”.

Install note: everything uses tools already on Kali except `hydra`. The brute-force commands use nmap's built-in scripts instead, so no install is needed; for hydra: `sudo apt install hydra`.